BDA 2-3-4-5 Practical Code

2.Develop a MapReduce program to calculate the frequency of a given word

```
from collections import defaultdict

# Mapper function
def mapper(text, search_word):
    mapped_data = []

    words = text.lower().split()

    for word in words:
        word = word.strip(".,!?;:()[]{}")

        if word == search_word.lower():
            mapped_data.append((word, 1))

    return mapped_data


# Shuffle and Sort
def shuffle_sort(mapped_data):
    grouped_data = defaultdict(list)

    for key, value in mapped_data:
        grouped_data[key].append(value)

    return grouped_data
```

```
# Reducer function
def reducer(grouped_data):
    result = {}

    for key in grouped_data:
        result[key] = sum(grouped_data[key])

    return result



# Main program
text = input("Enter text: ")
search_word = input("Enter word to search: ")

# Map phase
mapped = mapper(text, search_word)

print("\nMapped Output:")
print(mapped)

# Shuffle phase
grouped = shuffle_sort(mapped)

print("\nAfter Shuffle & Sort:")
print(dict(grouped))

# Reduce phase
final_result = reducer(grouped)
print("\nFinal Output:")
if final_result:
    for word, count in final_result.items():
        print(f"Frequency of '{word}' = {count}")
```

```
else:
    print("Word not found.")
```

3. Map-Reduce Matrix Multiplication

```
# Mapper function
def mapper(A, B, n):
    mapped_data = []

    for i in range(n):
        for j in range(n):
            for k in range(n):
                mapped_data.append(((i, j), A[i][k] * B[k][j]))

    return mapped_data


# Reducer function
def reducer(mapped_data):
    result = {}

    for key, value in mapped_data:
        if key not in result:
            result[key] = 0

        result[key] += value

    return result


# Input size
n = int(input("Enter size of matrix: "))
```

```
# Input first matrix
print("Enter first matrix:")
A = []

for i in range(n):
    row = list(map(int, input().split()))
    A.append(row)

# Input second matrix
print("Enter second matrix:")
B = []

for i in range(n):
    row = list(map(int, input().split()))
    B.append(row)

# Map phase
mapped = mapper(A, B, n)

print("\nMapped Output:")
print(mapped)

# Reduce phase
reduced = reducer(mapped)

# Print final matrix
print("\nResult Matrix:")

for i in range(n):
    for j in range(n):
```

```
    print(reduced[(i, j)], end=" ")
print()
```

4. MapReduce program to find the grades of students.

```
# Mapper function
def mapper(students):
    mapped_data = []

    for name, marks in students:

        if marks >= 90:
            grade = "A"

        elif marks >= 75:
            grade = "B"

        elif marks >= 60:
            grade = "C"

        else:
            grade = "D"

        mapped_data.append((name, grade))

    return mapped_data


# Reducer function
def reducer(mapped_data):
    result = {}

    for name, grade in mapped_data:
        result[name] = grade
```

```
    return result


# Input
n = int(input("Enter number of students: "))

students = []

for i in range(n):
    name = input("Enter student name: ")
    marks = int(input("Enter marks: "))

    students.append((name, marks))


# Map phase
mapped = mapper(students)

print("\nMapped Output:")
print(mapped)


# Reduce phase
result = reducer(mapped)

print("\nFinal Output:")

for name, grade in result.items():
    print(name, "-> Grade", grade)
```

5. MapReduce program to analyze Titanic ship data and to find the average age of the people (only male) who died in the tragedy. How many persons are dead in each class (only female).

```
# Mapper function
def mapper(passengers):
    mapped_data = []

    for gender, age, status, pclass in passengers:

        # Male dead
        if gender.lower() == "male" and status.lower() == "dead":
            mapped_data.append(("male_age", age))

        # Female dead
        if gender.lower() == "female" and status.lower() == "dead":
            mapped_data.append((f"class_{pclass}", 1))

    return mapped_data


# Reducer function
def reducer(mapped_data):
    male_age_sum = 0
    male_count = 0

    female_class_count = {}

    for key, value in mapped_data:

        if key == "male_age":
            male_age_sum += value
            male_count += 1
```

```
        else:
            if key not in female_class_count:
                female_class_count[key] = 0

            female_class_count[key] += value

    # Average age of dead males
    if male_count > 0:
        avg_age = male_age_sum / male_count
    else:
        avg_age = 0

    return avg_age, female_class_count


# Main program
n = int(input("Enter number of passengers: "))

passengers = []

print("\nEnter data in format:")
print("Gender Age Status Class")

for i in range(n):
    gender = input("Enter gender: ")
    age = int(input("Enter age: "))
    status = input("Enter status (dead/survived): ")
    pclass = input("Enter class (1/2/3): ")

    passengers.append((gender, age, status, pclass))
```

```
# Map phase
mapped = mapper(passengers)

print("\nMapped Output:")
print(mapped)



# Reduce phase
avg_age, female_deaths = reducer(mapped)

print("\nFinal Output:")
print("Average age of dead males =", avg_age)

print("\nFemale deaths by class:")
for cls, count in female_deaths.items():
    print(cls, "=", count)
```